

Algoritmi e strutture dati - esame del 16/9/2019

Svolgimenti

Esercizio 1. (Punti 3) Sia $f(n)$ una funzione non decrescente e positiva per qualunque n . Si dimostri oppure si refuti ognuna delle seguenti implicazioni:

- (1) $f(n) \in O(g(n)) \implies f(n) \notin \Omega(g(n))$
- (2) $f(n) \in O(g(n)) \implies f(n) \notin \Theta(g(n))$
- (3) $f(n) \notin O(g(n)) \implies f(n) \in \Theta(g(n))$

Soluzione 1. Le implicazioni (1)-(3) sono tutte false, ad esempio quando $f(n) = g(n)$.

Esercizio 2. (Punti 5) Si analizzi la complessità temporale in funzione di n del seguente algoritmo, ricavando la relazione di ricorrenza e calcolandone il Θ :

```
BUMM( $n$ )
  if  $n \leq 1$  then return 5
  else
     $m \leftarrow 0$ 
    for  $i \leftarrow 1$  to 4 do
       $m \leftarrow m + 2 \cdot \text{BUMP}(\lfloor n/2 \rfloor)$ 
    for  $i \leftarrow 1$  to  $n$  do
      for  $j \leftarrow 1$  to  $n$  do
         $m \leftarrow m + 1$ 
    return  $m$ 
```

Soluzione 2. La relazione di ricorrenza di BUMM è $T(n) = 4T(n/2) + n^2$. Svolgendo si trova:

$$\begin{aligned} T(n) &= 4T(n/2) + n^2 \\ &= 4(4T(n/4) + n^2/4) + n^2 = 4^2T(n/2^2) + 2n^2 \\ &= 4^kT(n/2^k) + kn^2 \end{aligned}$$

ovvero:

$$T(n) = 2^{2k}T(n/2^k) + kn^2 \quad k \leq \log n$$

Essendo $2^{2k} = (2^k)^2$, ne segue che:

$$T(n) = (2^{\log n})^2 + n^2 \log n = n^2 + n^2 \log n \in \Theta(n^2 \log n).$$

Esercizio 3. (Punti 8) Si consideri il seguente algoritmo:

```

BUBY( $A[0..n-1]$ )
  Pre:  $A$  è un array di interi passato per riferimento
  for  $i \leftarrow n-1$  downto 0 do
    if  $A[i] \bmod 2 = 0$  then
       $temp \leftarrow A[i]$ 
      for  $j \leftarrow i$  to  $n-2$  do
         $A[j] \leftarrow A[j+1]$ 
       $A[n-1] \leftarrow temp$ 

```

Si forniscano:

1. la post-condizione di $BUBY(A)$, ossia si dica come è stato modificato il contenuto dell'array A ;
2. un confine superiore stretto alla sua complessità in termini di O -grande;
3. un algoritmo equivalente a $BUBY$ a meno della posizione relativa degli elementi con la stessa parità, che sia asintoticamente ottimo.

Si giustifichino le risposte, fornendo le opportune descrizioni o argomentazioni.

Soluzione 3. La post-condizione di $BUBY$ è che l'array risultante $A'[0..n-1]$ è una permutazione di quello dato, ed esiste $k \in 0..n-1$ tale che gli elementi di $A'[0..k-1]$ siano tutti dispari, mentre quelli di $A'[k..n-1]$ siano tutti pari.

La complessità di $BUBY$ è dell'ordine di $O(\sum_{i=1}^n i) = O(n^2)$.

Esiste un algoritmo equivalente, ma di complessità $O(n)$ e quindi ottimo per la dimensione dei dati:

```

OPTIMALBUBY( $A[0..n-1]$ )
   $k \leftarrow n$ 
  for  $i \leftarrow n-1$  downto 0 do // inv.  $\forall x \in A[k..n-1]. x \bmod 2 = 0 \wedge \forall y \in A[i+1..k-1]. y \bmod 2 = 1$ 
    if  $A[i] \bmod 2 = 0$  then
       $k \leftarrow k-1$ 
      scambia  $A[i]$  con  $A[k]$ 

```

Esercizio 4. (Punti 5) Si consideri lo heap minimo rappresentato con l'array

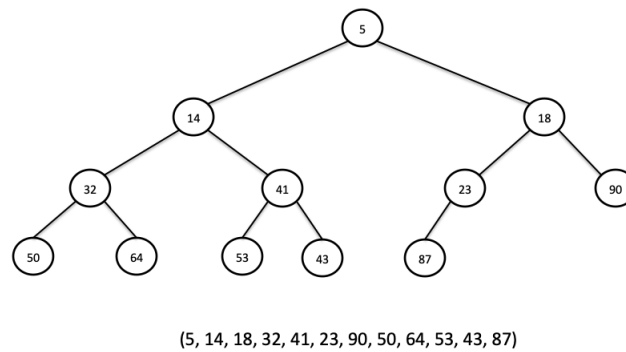
(5, 14, 18, 32, 41, 23, 90, 50, 64, 53, 43, 87)

Si effettuino le seguenti operazioni una dopo l'altra:

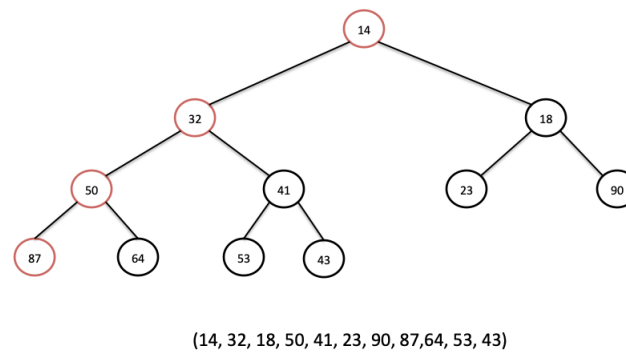
- estrazione del minimo
- inserimento della chiave 15

Si disegni lo heap risultante dopo ciascuna operazione (non dopo ciascuno scambio).

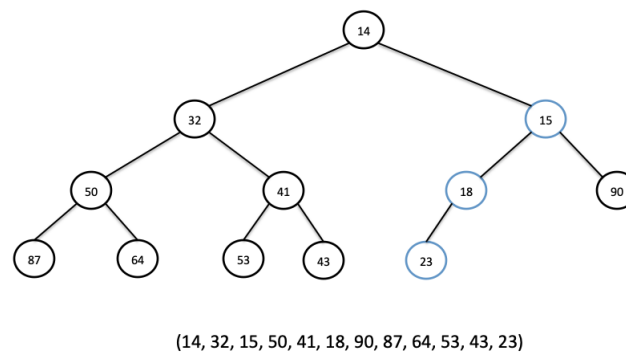
Soluzione 4. Lo heap dato risulta:



Dopo l'estrazione del minimo, ossia di 5, lo heap risulta:



Infine lo heap dopo l'inserimento della chiave 15 risulta:



Esercizio 5. (Punti 6) Sia dato un albero k -ario T non vuoto, rappresentato con puntatori *child* e *sibling*. Si dia un algoritmo che determini la larghezza dell'albero, cioè il numero massimo di nodi che giacciono sul medesimo livello, con complessità $O(n)$.

Soluzione 5. Una soluzione consiste in una BFS, dove la coda contiene coppie (x, n) dove x è un (puntatore ad un) nodo di livello n :

```

LARGE( $T$ )
  Pre:  $T \neq nil$ 
   $Q \leftarrow \emptyset$  // coda vuota
  ENQUEUE( $(T, 0), Q$ )
   $level \leftarrow 0$  // livello corrente
   $levlarge \leftarrow 1$  // larghezza del livello corrente
   $maxlarge \leftarrow 1$  // massimo delle larghezze
  while  $Q \neq \emptyset$  do
     $(x, n) \leftarrow$  DEQUEUE( $Q$ )
    if  $n > level$  then
       $maxlarge \leftarrow \max(maxlarge, levlarge)$ 
       $level \leftarrow n, levlarge \leftarrow 1$ 
    else //  $n = level$ 
       $levlarge \leftarrow levlarge + 1$ 
     $C \leftarrow x.child$ 
    while  $C \neq nil$  do
      ENQUEUE( $(C, n + 1), Q$ )
       $C \leftarrow C.sibling$ 
  return  $\max(levlarge, maxlarge)$  // per tener conto anche dell'ultimo livello

```

Esercizio 6. (Punti 4) Si riporti l'algoritmo INSERT(k, T) di inserimento di una chiave k in un albero binario di ricerca (in breve ordinato) T , eventualmente vuoto. Per semplicità si assuma che T sia realizzato con i soli puntatori *left* e *right*, ossia senza il puntatore al padre.

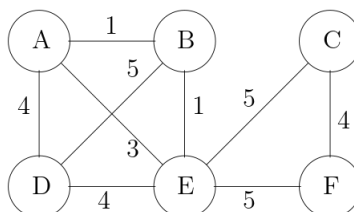
Soluzione 6.

```

INSERT( $k, T$ )
  Pre:  $T$  è ordinato ed eventualmente vuoto
  Post:  $T$  è ordinato e contiene la chiave  $k$ 
  if  $T = nil$  then
    return  $(k, nil, nil)$ 
  else
    if  $k = T.key$  then
      return  $T$ 
    else
      if  $k < T.key$  then
         $T.left \leftarrow$  INSERT( $k, T.left$ )
        return  $T$ 
      else
         $T.right \leftarrow$  INSERT( $k, T.right$ )
        return  $T$ 

```

Esercizio 7. (Punti 5) Si disegnino tutti gli alberi minimi ricoprenti del grafo seguente.



Soluzione 7. Gli MST, il cui peso è 15, sono i seguenti:

